

✓ Import Libraries

```
#@title Import Libraries

import pandas as pd
import numpy as np
import pandas as pd
from scipy.stats import mode
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from imblearn.over_sampling import RandomOverSampler
%matplotlib inline
```

✓ Loading dataset

```
#@title Loading dataset

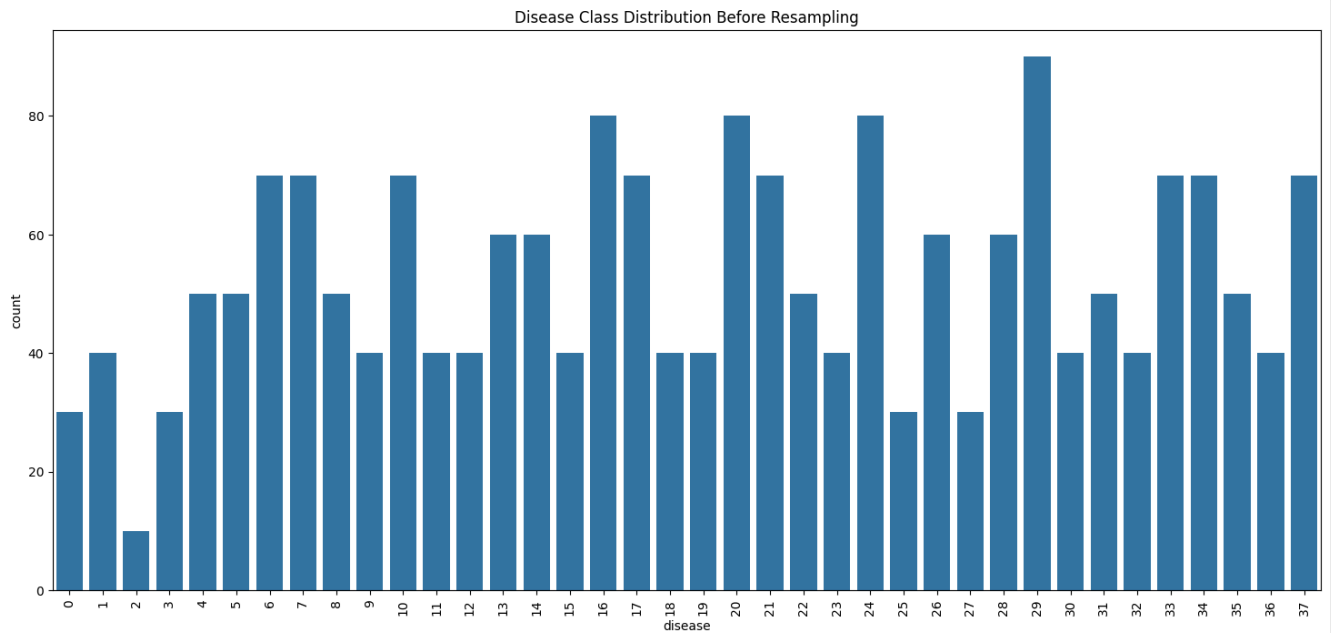
data = pd.read_csv('/content/drive/MyDrive/improved_disease_dataset.csv')

encoder = LabelEncoder()
data["disease"] = encoder.fit_transform(data["disease"])

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

plt.figure(figsize=(18, 8))
sns.countplot(x=y)
plt.title("Disease Class Distribution Before Resampling")
plt.xticks(rotation=90)
plt.show()

ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)
```



✓ Using Stratified K-fold Cross-Validation on 3 ML Models

```
#@title Using Stratified K-fold Cross-Validation on 3 ML Models
```

```
# Label encode 'gender' column if it exists
if 'gender' in X_resampled.columns:
    le = LabelEncoder()
    X_resampled['gender'] = le.fit_transform(X_resampled['gender'])

# Fill any remaining NaN values with 0
X_resampled = X_resampled.fillna(0)

# Flatten y_resampled if it's not already a 1D array
if len(y_resampled.shape) > 1:
    y_resampled = y_resampled.values.ravel()

# Define the models to evaluate
models = {
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC()
}

# Set the scoring metric and Stratified K-fold parameters
cv_scoring = 'accuracy' # you can also use 'f1_weighted', 'roc_auc_ovr' for multi-class
stratified_kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluate each model using cross-validation
for model_name, model in models.items():
    try:
        scores = cross_val_score(
            model,
            X_resampled,
            y_resampled,
            cv=stratified_kfold,
            scoring=cv_scoring,
            n_jobs=-1,
            error_score='raise'
        )
```

```

        print("=" * 50)
        print(f"Model: {model_name}")
        print(f"Scores: {scores}")
        print(f"Mean Accuracy: {scores.mean():.4f}")
    except Exception as e:
        print("=" * 50)
        print(f"Model: {model_name} failed with error:")
        print(e)

```

```

=====
Model: Decision Tree
Scores: [0.5497076  0.54239766 0.5380117  0.53654971 0.52923977]
Mean Accuracy: 0.5392
=====
Model: Random Forest
Scores: [0.54824561 0.5380117  0.54239766 0.53216374 0.51900585]
Mean Accuracy: 0.5360
=====
Model: SVM
Scores: [0.51315789 0.51169591 0.49707602 0.48245614 0.49561404]
Mean Accuracy: 0.5000

```

✓ Training Individual Models and Grnerating Confusion matrix

✓ Numerical labels - Actual disease names

```

#@title Numerical labels - Actual disease names
# Display the mapping between numerical labels and disease names
print("Mapping of Numerical Labels to Disease Names:")
for i, disease_name in enumerate(encoder.classes_):
    print(f"{i}: {disease_name}")

```

```

Mapping of Numerical Labels to Disease Names:
0: AIDS
1: Acne
2: Alcoholic hepatitis
3: Allergy
4: Arthritis
5: Bronchial Asthma
6: Cervical spondylosis
7: Chronic cholestasis
8: Dengue
9: Diabetes
10: Dimorphic hemorrhoids(piles)
11: Drug Reaction
12: GERD
13: Gastroenteritis
14: Heart attack
15: Hepatitis A
16: Hepatitis B
17: Hepatitis C
18: Hepatitis D
19: Hepatitis E
20: Hypertension
21: Hyperthyroidism
22: Hypoglycemia
23: Hypothyroidism
24: Impetigo
25: Jaundice
26: Malaria
27: Migraine
28: Osteoarthritis
29: Paralysis (brain hemorrhage)

```

30: Peptic ulcer disease
 31: Pneumonia
 32: Psoriasis
 33: Tuberculosis
 34: Typhoid
 35: Urinary tract infection
 36: Varicose veins
 37: Vertigo (Benign paroxysmal Positional Vertigo)

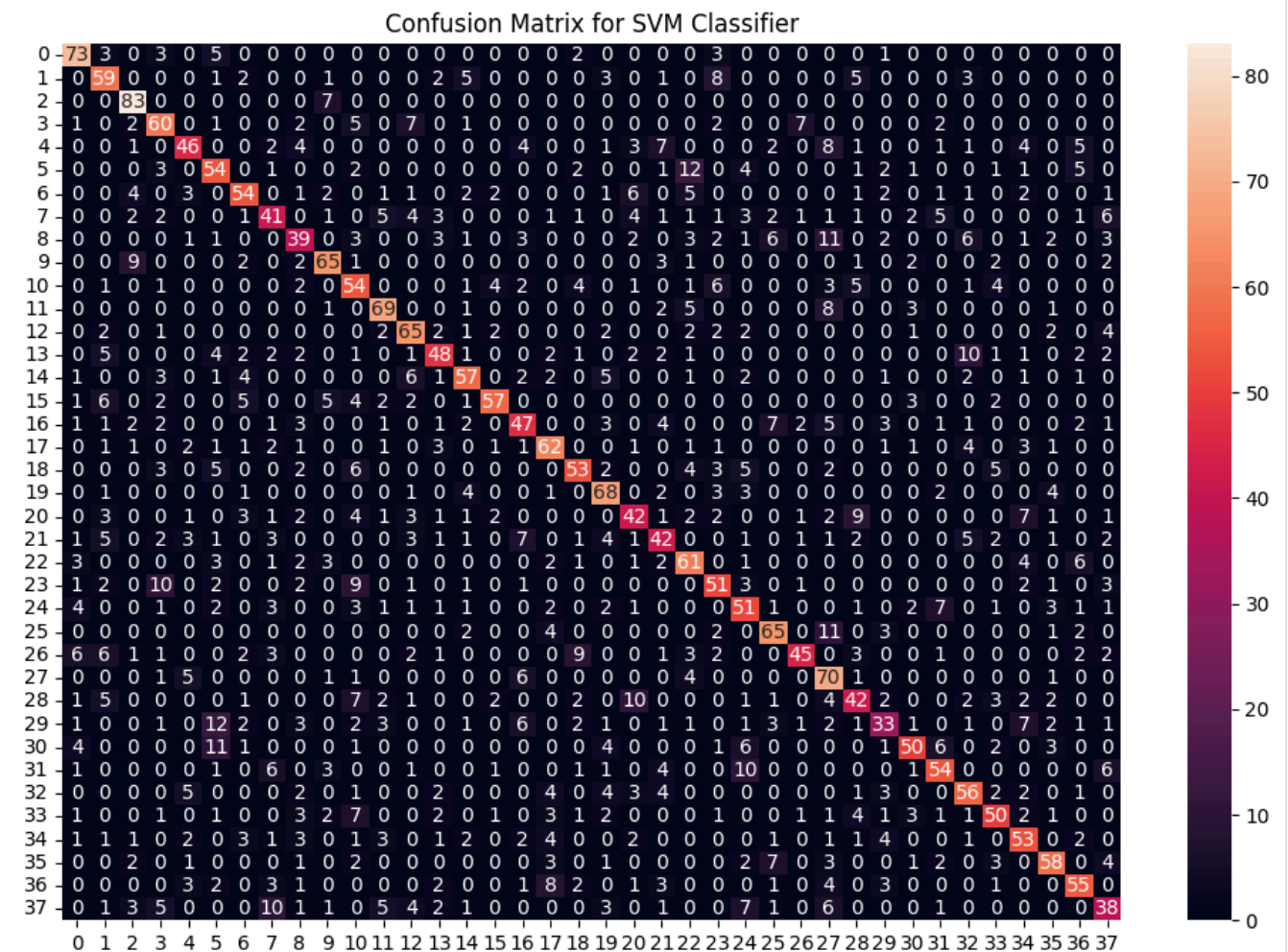
✓ Support Vector Classifier

```
#@title Support Vector Classifier
```

```
svm_model = SVC()
svm_model.fit(X_resampled, y_resampled)
svm_preds = svm_model.predict(X_resampled)
```

```
cf_matrix_svm = confusion_matrix(y_resampled, svm_preds)
plt.figure(figsize=(12, 8))
sns.heatmap(cf_matrix_svm, annot=True, fmt="d")
plt.title("Confusion Matrix for SVM Classifier")
plt.show()
```

```
print(f"SVM Accuracy: {accuracy_score(y_resampled, svm_preds) * 100:.2f}%")
```



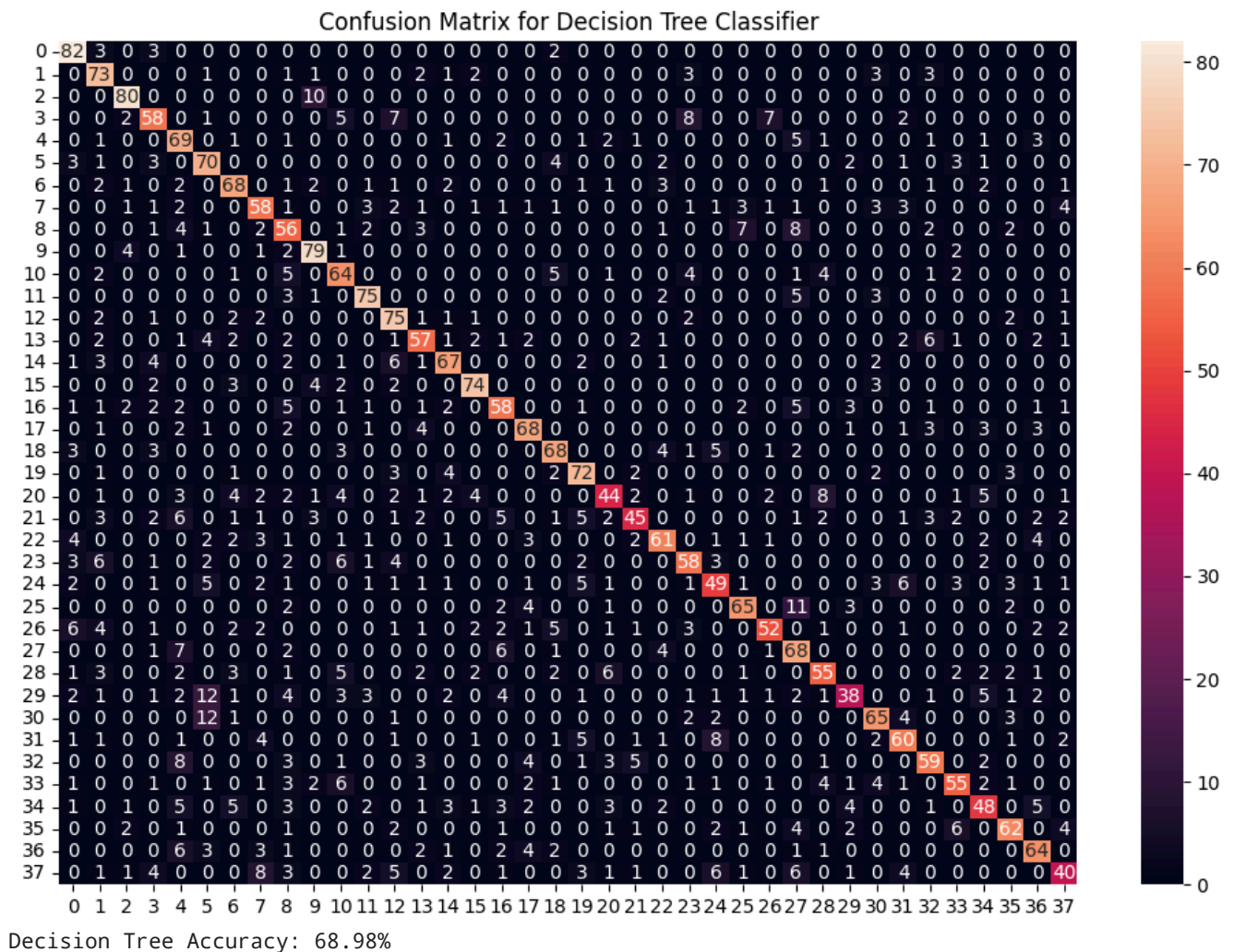
Decision Tree Classifier

```
#@title Decision Tree Classifier

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_resampled, y_resampled)
dt_preds = dt_model.predict(X_resampled)

cf_matrix_dt = confusion_matrix(y_resampled, dt_preds)
plt.figure(figsize=(12, 8))
sns.heatmap(cf_matrix_dt, annot=True, fmt="d")
plt.title("Confusion Matrix for Decision Tree Classifier")
plt.show()

print(f"Decision Tree Accuracy: {accuracy_score(y_resampled, dt_preds) * 100:.2f}%")
```



Random Forest Classifier

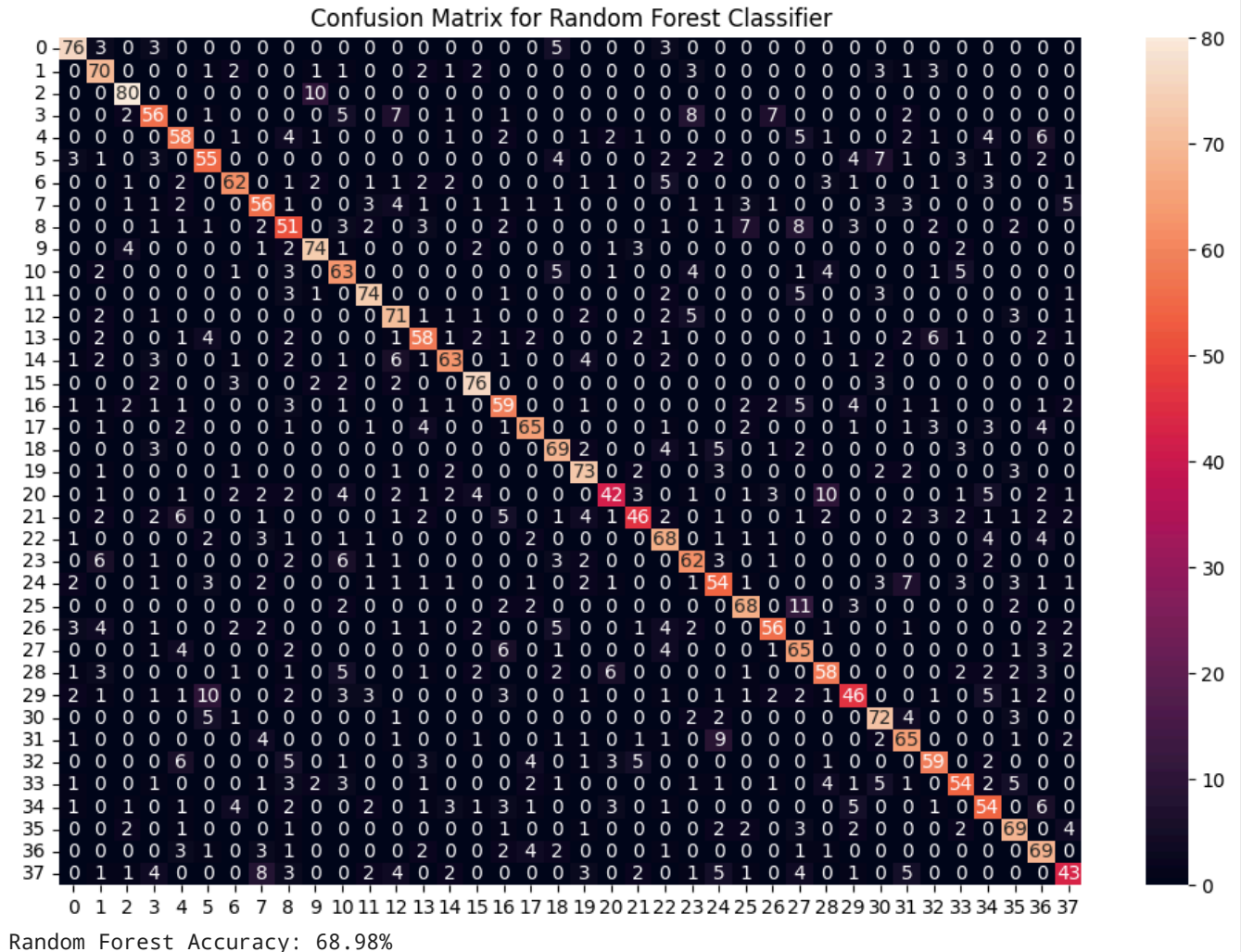
```
#@title Random Forest Classifier

rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_resampled, y_resampled)
rf_preds = rf_model.predict(X_resampled)

cf_matrix_rf = confusion_matrix(y_resampled, rf_preds)
```

```
plt.figure(figsize=(12, 8))
sns.heatmap(cf_matrix_rf, annot=True, fmt="d")
plt.title("Confusion Matrix for Random Forest Classifier")
plt.show()

print(f"Random Forest Accuracy: {accuracy_score(y_resampled, rf_preds) * 100:.2f}%")
```



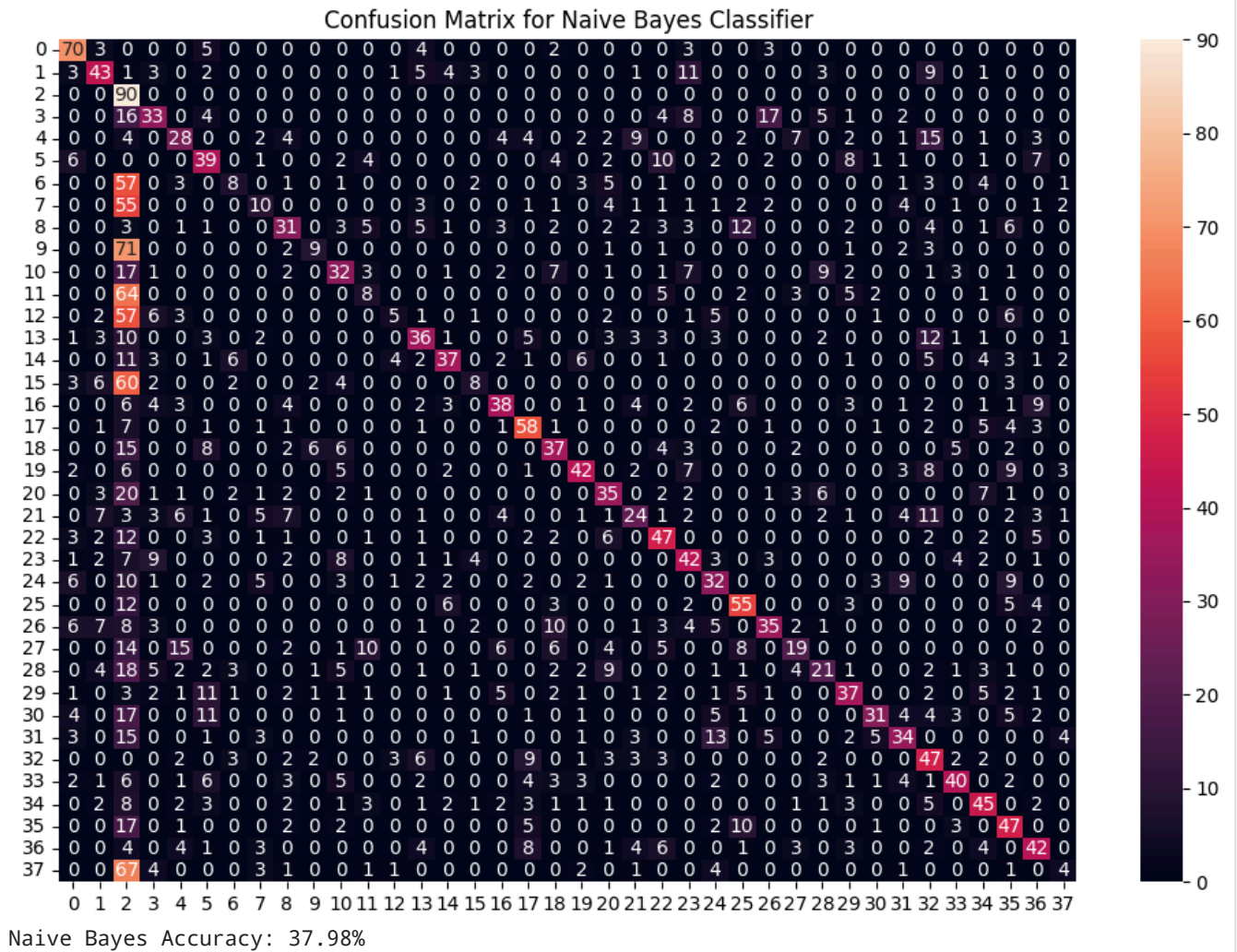
▼ Trying Gaussian Naive Bayes

```
#@title Trying Gaussian Naive Bayes

nb_model = GaussianNB()
nb_model.fit(X_resampled, y_resampled)
nb_preds = nb_model.predict(X_resampled)

cf_matrix_nb = confusion_matrix(y_resampled, nb_preds)
plt.figure(figsize=(12, 8))
sns.heatmap(cf_matrix_nb, annot=True, fmt="d")
plt.title("Confusion Matrix for Naive Bayes Classifier")
plt.show()

print(f"Naive Bayes Accuracy: {accuracy_score(y_resampled, nb_preds) * 100:.2f}%")
```



Combining all 4 Models for prediction

```
#@title Combining all 4 Models for prediction

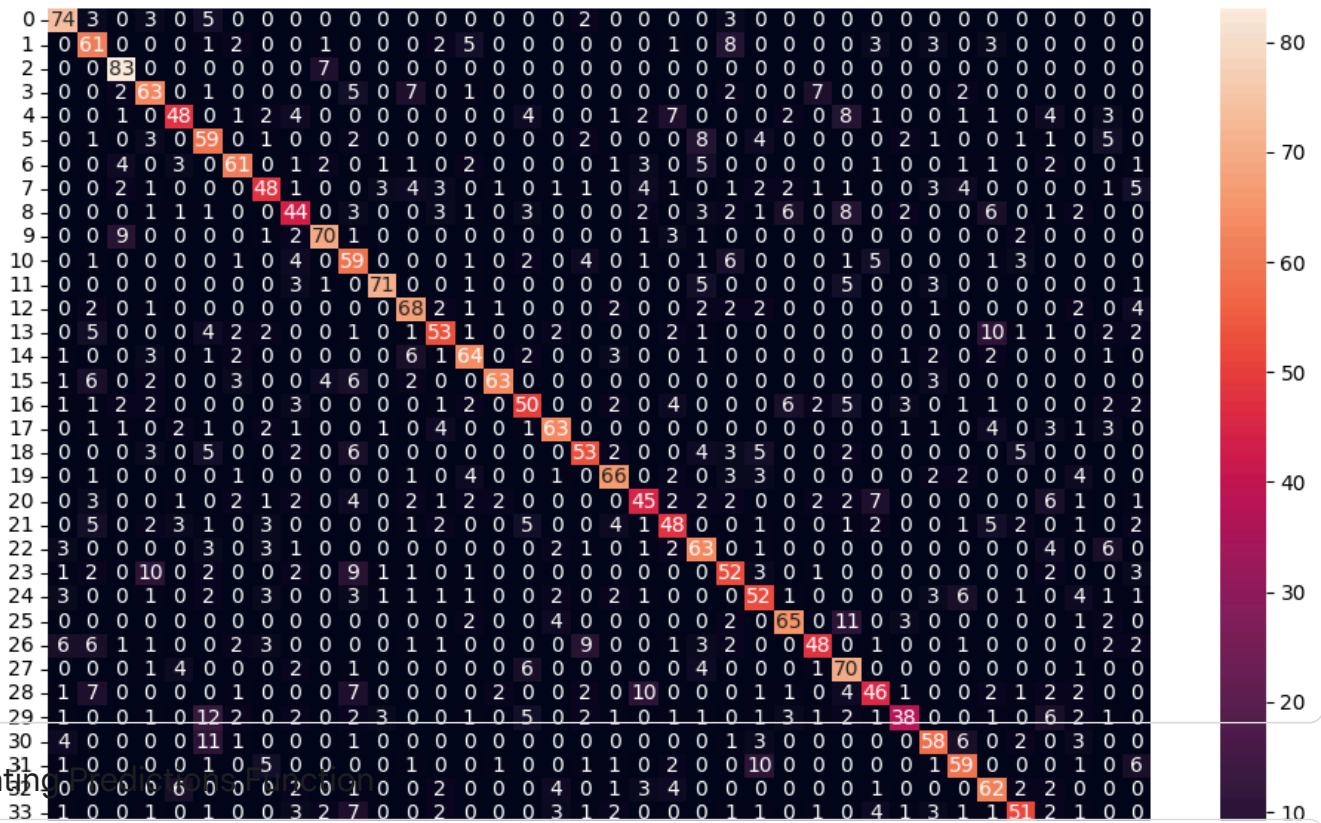
from statistics import mode

final_preds = [mode([i, j, k, l]) for i, j, k, l in zip(svm_preds, nb_preds, rf_preds, dt_preds)]

cf_matrix_combined = confusion_matrix(y_resampled, final_preds)
plt.figure(figsize=(12, 8))
sns.heatmap(cf_matrix_combined, annot=True, fmt="d")
plt.title("Confusion Matrix for Combined Model")
plt.show()

print(f"Combined Model Accuracy: {accuracy_score(y_resampled, final_preds) * 100:.2f}%")
```

Confusion Matrix for Combined Model



Creating Predictions Function

```
#@title Creating Predictions Function

symptoms = X.columns.values
symptom_index = {symptom: idx for idx, symptom in enumerate(symptoms)}

def predict_disease(input_symptoms):
    input_symptoms = input_symptoms.split(",")
    input_data = [0] * len(symptom_index)

    for symptom in input_symptoms:
        if symptom in symptom_index:
            input_data[symptom_index[symptom]] = 1

    input_df = pd.DataFrame([input_data], columns=symptoms)

    rf_pred = encoder.classes_[rf_model.predict(input_df)[0]]
    nb_pred = encoder.classes_[nb_model.predict(input_df)[0]]
    svm_pred = encoder.classes_[svm_model.predict(input_df)[0]]
    dt_pred = encoder.classes_[dt_model.predict(input_df)[0]]

    final_pred = mode([rf_pred, nb_pred, svm_pred, dt_pred])

    return {
        "Random Forest Prediction": rf_pred,
        "Naive Bayes Prediction": nb_pred,
        "SVM Prediction": svm_pred,
        "Decision Tree Prediction": dt_pred,
        "Final Prediction": final_pred
    }

print(predict_disease("fever,cough,headache,vomiting, fatigue"))

{'Random Forest Prediction': 'Arthritis', 'Naive Bayes Prediction': 'Hyperthyroidism', 'SVM Pred
```

```
# Display the list of symptoms  
print("List of Symptoms in the Dataset:")  
print(symptoms)
```

```
List of Symptoms in the Dataset:  
['fever' 'headache' 'nausea' 'vomiting' 'fatigue' 'joint_pain' 'skin_rash']
```